

PRACTICE: Homework 2

Due: Friday, April 14, at 11:59pm

In this assignment you'll gain some more experience with shell commands and with writing short shell scripts.

General hint: Before trying to write and debug a script, it's very helpful to first work out the necessary commands by experimenting in a shell window. Similarly, you can test one statement in your script at a time in your shell. Also, look at man pages and other descriptions of commands. Sometimes options are available that allow a single command to do something you want instead of having to use several commands or writing loops or complex control structures in a script. You can also find ways to control how much output a command gives.

Requirement Specifications

1. (Script) Create a bash script `combine` that takes 2 or more arguments, call them `f1`, `f2`, ..., `fn`.

Script `combine` should work as follows:

- All arguments are treated as filenames.
- If fewer than two arguments are given, print `Usage: combine outputfilename [inputfilename ...]` error message on `stderr` and exit with a return code of 1.
- If a file or directory `f1` already exists, print `Error: Output file should not exist on stderr` and exit with a return code of 1.
- Otherwise concatenate the contents of `f2`, ..., `fn` and put them in `f1`. You will want to handle `cat` errors with input files (for example, if a file does not exist), but do not print these error. Instead, these error messages should be redirected to `f1`. Exit with a return code of 0 after copying all the input files.

A simple example of running `combine` is:

```
[localhost]$ echo "making file 1" > file1
[localhost]$ echo "and another one for file 2" > file2
[localhost]$ touch file3
[localhost]$ ./combine output file1 file2 nonfile file3
[localhost]$ cat output
making file 1
and another one for file 2
cat: nonfile: No such file or directory
```

Restriction: You may not use the file names `/dev/stdout` or `/dev/stderr`. These are not

portable across *nix systems. Although they are found on most versions of Linux the problem can be solved without them.

Hint: Put filenames in double-quotes in case they contain "funny characters" (such as spaces). Your script should work with any file names, no matter what they contain.

Hint: What happens if you run `cat 'nonexisting file' ?`

Hints: `cat`, `shift`, `$@`, `-lt`, `-a`, `&>> .`

Hint: Hints are just ideas you might find useful, not things that must appear in your solution.

2. (Script) Create a bash script called `spellcheck` that takes one or more arguments, `f1`, `f2`, ..., `fn`

Script `spellcheck` should work as follows:

- If the script is not supplied with an input argument, print a usage note to `stderr` and exit with a code of 1

```
[localhost]$ spellcheck
Usage: spellcheck filename ...
```

- If an argument is supplied that is not a valid file (`[ ! -f $1 ]`), `spellcheck` prints an appropriate error message to `stderr` and skips that argument.
- Your script should read each word in a supplied file and use `grep` to compare that word to the dictionary (which is assumed to exist at `/usr/share/dict/words`). If the word is *not* found in the dictionary it should be added to the end of a file called `<FILE>.spelling.*` . You should add repeated words to the word list in the initial step, and you'll want to add words in the order you encounter them.
- If `<FILE>.spelling` does not exist, print a message to `stdout` stating that the script is **creating** it.
- If `<FILE>.spelling` already exists, print a message to `stdout` stating that you are deleting the old file and **replacing** it.
- The `grep` search should be case-insensitive.
- At the end of processing each file, print a message to `stdout` asserting completion of the file, with the number of words found, and the number of unique words found.
- Your script should exit with a return code of 0 after completing the final print statement.

For example, executing: `./spellcheck longtext` will create a file called `longtext.spelling` , which contains a list of words such as 'gnu.org' and 'Korn'. Example output is here:

```
$ ./spellcheck shorttext
./spellcheck creating shorttext.spelling file
./spellcheck processed shorttext and found 1 spelling errors 1 of which are un
$ ./spellcheck tricky shorttext
./spellcheck error: tricky does not exist - skipping.
./spellcheck replacing shorttext.spelling file
./spellcheck processed shorttext and found 1 spelling errors 1 of which are un
```

Hints:

- You should use `shorttext` (`shorttext`) and `longtext` (`longtext`) to test your code. The `longtext` file has 9 spelling errors 8 of which are unique.
- You can download these files to a bash shell using the command `wget https://courses.cs.washington.edu/courses/cse374/23sp/assignments/FILENAME`
- This is **not** a true spell check - you will identify any word that does not appear precisely in the dictionary. This includes numbers, words with weird punctuation, names, other edge cases. The provided files above have most of this removed for clarity.
- *If you want to try something more realistic, have a go at `linus.txt` (`linus.txt`). Your code will not be tested on this file, since implementation decisions must be made to approach a real spell-check. (For example, do numbers count? What about different forms of a word, or hyphenated words?)*
- Your `combine` script should provide a method for stepping through input arguments that you can re-use here.
- Nested loops allow you to read lines, and words within lines. This is not the only way to solve the problem, but it works.
- In Lecture 6 we learned how to use `grep` to find words in a file. Look for options that allow case insensitive and quiet operation. You may also be interested in `wc`, `sort`, and `uniq`.
- There are a couple of slides from Lecture 3 that talk about redirecting your output.
- As always, get one small part of your script working at a time.

Assessment

- This homework will be worth a total of 40 points. Each script is worth 20 points.
- Identifying information including your name, CSE 374 Homework 2, the problem number, and the date should appear as comments in each of your files. This will be graded.
- You should test for correct scripts, etc., that run with `bash` on Seaside.
- Use good style, including indentation and line breaks. Refer to the CSE 374 guide (https://courses.cs.washington.edu/courses/cse374/23sp/resources/bash_style_guide.html) for hints on style. This will be graded.
- Good style also includes some commenting, including your name, brief description of script purpose, and often a comment ahead of major code blocks describing their operation. This will be graded.

- Scripts should handle arguments with spaces in them, and handle errors appropriately with messages and graceful exits.
- Error messages should be sent to stderr, unless otherwise noted. If the error message isn't prescribed in the assignment, you should choose something reasonable and informative - using standard bash command output is a good idea.

Turning In

Please submit your files to Gradescope, which can be reached through Canvas or directly. You will submit two files, called combine and spellcheck. These file names do not have extensions on them.

There will be an autograder for these assignments, with additional manual checking for a few points. Students may re-submit to the autograder to improve their scores.

Please carefully check the requirements before submitting to the autograder.
